

challenge defines the essence of TinyML, where extreme resource constraints make it a unique engineering endeavour. Most microcontrollers have only 256KB to 1MB of flash memory and even less RAM, leaving minimal space for neural networks. They also lack powerful GPUs or multi-core CPUs, relying instead on simple processors running at low clock speeds. Power efficiency becomes critical, as many devices operate on small batteries, and every extra computation reduces their lifespan. Moreover, these devices must remain functional even without constant network connectivity. Updating models across thousands of distributed devices adds further complexity. These challenges aren't obstacles but design constraints that inspire innovation—precisely where open source frameworks empower developers to build efficient, deployable TinyML solutions.

Open source frameworks that make TinyML possible

A few years ago, if you wanted to run ML on a microcontroller, you were mostly on your own. Today, thanks to the open source community, there are some incredibly capable frameworks that take care of the heavy lifting.

TensorFlow Lite for

Microcontrollers (TFLM): TFLM is a trimmed-down version of TensorFlow Lite built specifically for microcontrollers. It runs without an OS and supports common layers like Conv2D, Fully Connected, and Softmax. Developers use it for things like wake-word detection, gesture recognition, and anomaly detection. It's battle-tested and widely supported.

Edge Impulse: This is one of my personal favourites because it's end-to-end. Edge Impulse gives you tools to collect and label data, train models, optimise them, and finally deploy to MCUs -- all in a developer-friendly interface. It works smoothly with Arduino, ESP32, STM32, and others. For someone who wants to build fast

without reinventing everything, this platform is a game changer.

MicroTVM: This one is for people who like fine-grained control. It's part of the Apache TVM project and lets you compile neural networks into extremely efficient C code for embedded targets. It's not plug-and-play like Edge Impulse, but the optimisation possibilities are excellent.

PyTorch Mobile / Lite Interpreter:

Although not exactly 'tiny', PyTorch Mobile works well for Raspberry Pi or Android-based edge devices. If your project allows for a bit more processing power, this is a flexible choice.

Building a TinyML application usually follows a clear sequence. Once you do it a couple of times, it starts to feel natural. The steps are:

1. Data collection and preprocessing
2. Model training
3. Model optimisation
4. Deployment

Real-world examples that show TinyML's power

Predictive maintenance in factories:

Imagine small vibration sensors attached to motors. A TinyML model learns the 'normal' vibration pattern. If something deviates, the device instantly flags it — no round-trip to the cloud, no delay.

Wake-word detection: You've probably used this without realising it. Many smart speakers use a tiny model running locally to detect a wake word. The model's job is simply to 'wake up' the larger system when needed, saving energy and ensuring privacy.

Wearable health monitoring:

TinyML models can run on wristbands or patches to detect irregular motion, falls, or heart anomalies. Because everything happens on-device, user data

stays private, and battery life improves dramatically.

Making TinyML devices part of the IoT network

TinyML devices use specialised IoT protocols to communicate reliably.

- **MQTT** is the go-to lightweight protocol. A TinyML device can simply publish a message like 'anomaly detected' to an MQTT broker.
- **Node-RED** gives you a nice flow-based way to wire these messages into dashboards, alerts, or other services.
- **ThingsBoard** and similar platforms act as the control centre -- receiving data, visualising it, and letting you trigger actions.

TinyML's role in the shift towards agentic AI

Here's the interesting part: TinyML isn't just about compressing models. It's about giving devices autonomy.

Right now, a single TinyML model can classify or detect anomalies. But imagine a network of these devices, each running its own models, collaborating to sense, decide, and act. That's not science fiction; it's the foundation for agentic AI at the edge. Instead of one big brain in the cloud, you get thousands of small, distributed brains working together.

Whether you're a hobbyist building something cool with Arduino or an engineer working on industrial systems, TinyML gives you a way to embed intelligence right where the data is born. And as these devices evolve, they'll form the building blocks for the next wave of intelligent, agentic systems — systems that sense, plan, and act without waiting for instructions. **END** 🐼

By: Abhishek Raj Permani

The author has a Master's degree in data science and AI. He has worked in companies like Jaguar Land Rover and Samsung Research, and is passionate about solving real-world problems in areas such as autonomous systems, transformer models, and recommendation engines. He frequently shares his knowledge at national conferences and forums.